

HEALTHCARE INFUSION ANALYTICS PIPELINE

Technical Documentation & Reproducible Workflow

Python | Pandas | DuckDB | SQL | XLSB

PROJECT PURPOSE

Transform multi-sheet infusion operations data into a validated analytical fact model and reproducible reporting layer for patient volume, treatment activity, drug/location utilization, weekly operations, and data-quality monitoring.

Prepared by Aryan Sharma

Portfolio Project Documentation

1. Executive Summary

This project builds an end-to-end analytical workflow for infusion-center operations. A source Excel Binary Workbook (.xlsb) is inspected and ingested into DuckDB, relationships are validated, a reusable fact_infusions analytical table is constructed, and SQL-driven reports are exported to CSV for downstream analysis or BI consumption.

The design deliberately separates raw ingestion, transformation, validation, modeling, and reporting. That separation makes the workflow easier to audit, reproduce, extend, and explain than a one-off spreadsheet analysis.

CORE ANALYTICAL DEFINITION

A successfully administered treatment is defined as meds_administered = 'Yes'. The source does not contain an actual administered-volume field, so the count of administered infusion events is used as the available proxy when answering drug-volume questions.

1.1 Project Deliverables

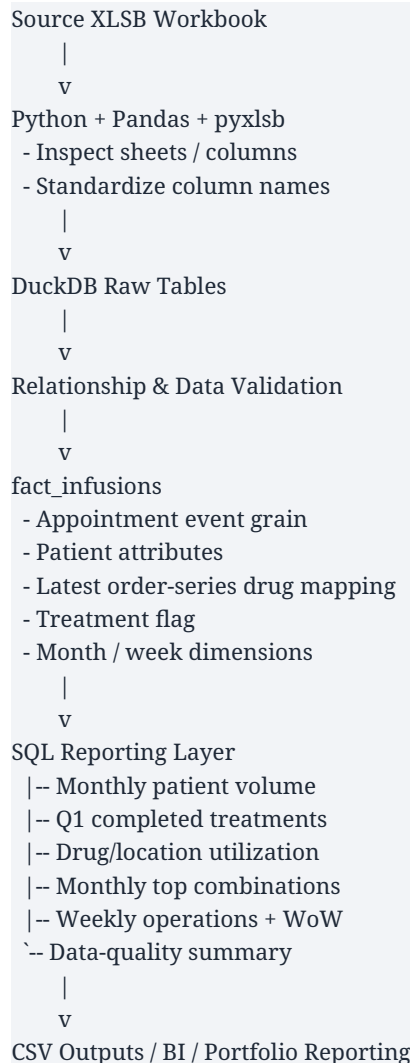
- A local DuckDB analytical database created from the source XLSB workbook.
- A curated fact_infusions table at the appointment/infusion-event grain.
- Join and relationship diagnostics across appointments, patients, order series, and authorization data.
- Six reproducible analytical outputs exported as CSV files.
- A documented set of business rules, assumptions, limitations, and extension opportunities.

1.2 Observed Case-Study Results

Measure	Result
Q1 2026 successfully administered treatments	1041
Highest overall drug/location combination	LEQEMBI - Sarasota (129 administered infusions)
Rows in analytical fact table	3780
Weekly report rows	493

2. Solution Architecture

The pipeline is intentionally lightweight: Python performs workbook inspection and ingestion, DuckDB provides the analytical SQL engine, and CSV outputs provide a portable handoff format for validation, reporting, or dashboarding.



2.1 Why DuckDB?

- Runs locally with no server administration or cloud database dependency.
- Supports analytical SQL, CTEs, window functions, date transformations, and pandas interoperability.
- Stores the complete case-study model in a single portable database file.
- Makes the project simple for another analyst or reviewer to reproduce on a laptop.

2.2 Modeling Philosophy

Instead of rebuilding joins inside every business query, the workflow creates a reusable analytical fact table first. The reporting layer then queries a stable model. This is closer to a production analytics pattern than repeatedly joining raw workbook sheets for each metric.

3. Source Data Inspection and Ingestion

3.1 Inspect the XLSB Workbook

The `inspect_xlsb.py` script opens the workbook with pandas using the `pyxlsb` engine, lists every worksheet, prints the columns and shape of each sheet, and shows sample records. This is the first profiling step and prevents assumptions about workbook structure from being embedded prematurely in transformations.

```
xls = pd.ExcelFile(file_path, engine='pyxlsb')
for sheet in xls.sheet_names:
    df = pd.read_excel(file_path, sheet_name=sheet, engine='pyxlsb')
    print(df.columns)
    print(df.shape)
    print(df.head())
```

3.2 Load Every Sheet into DuckDB

The loader standardizes table and column names for SQL usability while preserving source content. Column names are lowercased, punctuation is normalized to underscores, and repeated underscores are collapsed. Importantly, no source rows or columns are intentionally dropped during ingestion.

```
def clean_name(name):
    name = str(name).strip().lower()
    name = name.replace('.', '_')
    name = re.sub(r'^a-z0-9_+', '_', name)
    name = re.sub(r'_+', '_', name)
    return name.strip('_')

# Create one DuckDB table per source sheet
CREATE OR REPLACE TABLE <sheet_name> AS
SELECT * FROM temp_df
```

3.3 Inspect the Database

`inspect_db.py` enumerates all DuckDB tables, describes each schema, and prints sample rows. This verifies that workbook ingestion succeeded and provides a quick way to inspect raw data without reopening Excel.

4. Relationship Validation Before Modeling

Before constructing the analytical model, the workflow tests key relationships and distribution assumptions. This is critical because a technically valid SQL join can still create duplicate rows or silently lose records.

Validation	Purpose
Appointments -> Patients	Measure matched and unmatched appointment-to-patient relationships.
Appointments -> Order Series	Understand row multiplication risk when patients have

Validation	Purpose
	multiple order-series records.
Order Series -> Authorization	Check authorization linkage after type normalization.
Medication-administered values	Confirm the actual categorical values used to define successful treatment.
Appointment-status values	Understand workflow statuses and potential status/medication inconsistencies.
Location values	Profile operating locations represented in appointment records.

4.1 Multi-Order-Series Risk

The validation code explicitly identifies patients with more than one order-series record and counts distinct drugs. This reveals a one-to-many relationship that could duplicate appointment rows if raw order-series data were joined directly.

4.2 Date and Drug Exploration

Additional profiling checks drug values, patients associated with multiple distinct drugs, the converted appointment date range, status versus medication-administered combinations, and authorization matching using numeric casts where required.

5. Building the fact_infusions Analytical Model

fact_infusions is the central analytical layer. The appointment record is treated as the infusion event, enriched with patient demographics and the latest available order-series information.

5.1 Grain

FACT TABLE GRAIN

One fact row represents one appointment/infusion event with a non-null appointment start date. This grain is protected by avoiding a direct one-to-many join to all order-series records.

5.2 Latest Order-Series Selection

Order-series records are ranked within each patient. The most recent record, with order_series_id as a secondary tie-breaker, is selected for downstream mapping.

```
ROW_NUMBER() OVER (
  PARTITION BY patient_id
  ORDER BY order_series_created_date DESC,
```

```
order_series_id DESC
) AS rn
```

5.3 Date Transformation

The workbook stores dates as Excel serial numbers. The transformation uses Excel's 1899-12-30 origin, then derives month and week dimensions for reporting.

```
DATE '1899-12-30' + CAST(appointments_start_date AS INTEGER) AS infusion_date
DATE_TRUNC('month', infusion_date) AS infusion_month
DATE_TRUNC('week', infusion_date) AS infusion_week
```

5.4 Treatment Flag

A reusable binary flag centralizes the successful-treatment rule so every downstream query uses the same definition.

```
CASE
  WHEN appointments_meds_administered = 'Yes' THEN 1
  ELSE 0
END AS medication_administered_flag
```

5.5 Final Fact Fields

Field Group	Examples
Keys	appointment_id, patient_id, order_series_id
Patient	age, sex
Dates	appointment_created_date, infusion_date, infusion_month, infusion_week
Operations	location, appointment_status, piv_attempts
Treatment	meds_administered, medication_administered_flag
Medication / Authorization	drug_name, auth_required, order_series_created_date

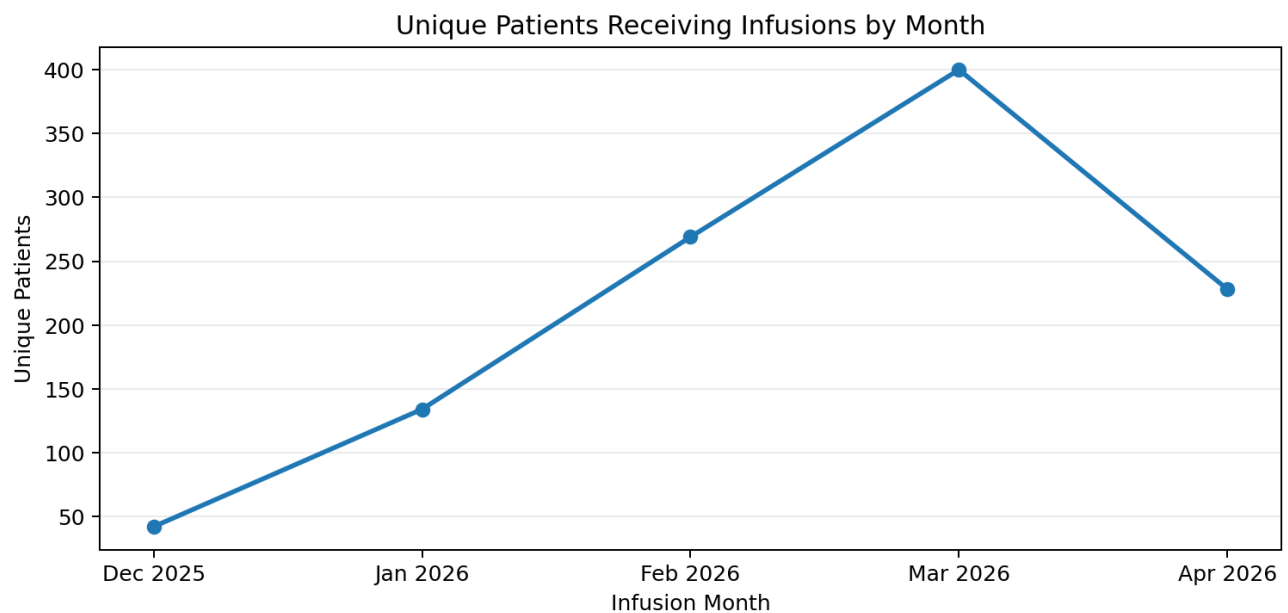
6. Analytical Queries and Business Logic

6.1 Unique Patients Receiving Infusions by Month

Counts distinct patient IDs for successfully administered treatments, grouped by infusion month. The metric answers how many unique patients actually received medication rather than how many appointments were scheduled.

```
SELECT infusion_month,
  COUNT(DISTINCT patient_id) AS unique_patients_received_infusions
FROM fact_infusions
```

```
WHERE medication_administered_flag = 1
GROUP BY infusion_month;
```



Month	Unique Patients
Dec 2025	42
Jan 2026	134
Feb 2026	269
Mar 2026	400
Apr 2026	228

6.2 Successfully Administered Treatments - Q1 2026

Using the defined medication-administered flag and a half-open date window from 2026-01-01 through 2026-04-01, the case dataset contains 1,041 successfully administered treatments in Q1 2026.

6.3 Highest Drug / Location Combination Overall

The highest observed combination is LEQEMBI at Sarasota, with 129 administered infusion events. This is a count-based utilization result, not an administered-dose or volume result.

6.4 Highest Drug / Location Combination by Month

Month	Drug	Location	Administered Infusions
Dec 2025	LEQEMBI	Fort Myers	6

Month	Drug	Location	Administered Infusions
Jan 2026	LEQEMBI	Sarasota	23
Feb 2026	LEQEMBI	Sarasota	31
Mar 2026	LEQEMBI	Sarasota	58
Apr 2026	LEQEMBI	The Villages	17

6.5 Weekly Operations Report

The weekly report aggregates by infusion week, location, and drug. It measures total administered infusions, unique patients, average PIV attempts, prior-week infusions, and week-over-week percentage change.

```
LAG(total_infusions) OVER (
  PARTITION BY location, drug_name
  ORDER BY infusion_week
) AS prior_week_infusions

100.0 * (total_infusions - prior_week_infusions)
/ prior_week_infusions AS wow_change_pct
```

WHY THE PARTITION MATTERS

Week-over-week comparison is performed within each location + drug combination, so the prior value is operationally comparable rather than coming from another site or medication.

7. Data Quality Controls

The project includes an explicit data-quality summary instead of assuming that operational statuses and clinical workflow fields are internally consistent.

Control	Observed Rows
Total fact rows	3780
Missing patient ID	0
Missing infusion date	0
Missing location	0

Control	Observed Rows
Missing drug	0
Missing PIV attempts	2435
Negative PIV attempts	0
Status = Complete but meds_administered = No	25
Status != Complete but meds_administered = Yes	2

INTERPRETATION

The most prominent completeness issue in the case dataset is PIV attempts: 2,435 of 3,780 fact rows are missing that field. The workflow preserves these nulls rather than silently imputing them. Status/medication mismatches are also surfaced for review.

8. Reproducing the Project Step by Step

1. Place the approved source XLSB workbook in the project data location (or update file_path in the ingestion scripts).
2. Install Python dependencies such as pandas, duckdb, and pyxlsb.
3. Run inspect_xlsb.py to understand sheets, columns, shapes, and sample values.
4. Run load_to_duckdb.py to create normalized raw DuckDB tables while retaining source rows and columns.
5. Run inspect_db.py to verify schemas and sample rows in the generated database.
6. Run check_joins.py and explore_drugs_dates.py to validate relationships, categories, date conversion, and one-to-many risks.
7. Run create_fact_infusions.py to build the curated analytical fact table.
8. Review fact rows and data-quality checks using view_fact_infusions.py and view_data_quality.py.
9. Run run_assessment_queries.py to execute the six analytical outputs and save each result to CSV.
10. Review results, document assumptions, and connect the output layer to a dashboard or downstream analytical workflow if desired.

8.1 Example Commands

```
python src/inspect_xlsb.py
python src/load_to_duckdb.py
python src/inspect_db.py
python src/check_joins.py
python src/explore_drugs_dates.py
python src/create_fact_infusions.py
python src/view_fact_infusions.py
python src/view_data_quality.py
python src/run_assessment_queries.py
```

8.2 Recommended requirements.txt

```
pandas
numpy
duckdb
pyxlsb
```

9. Recommended GitHub Repository Structure

Yes: after creating the project folder inside your existing Data-Science_projects repository, upload the project assets into that folder. Organize them so a reviewer can understand what is source code, SQL, generated output, and documentation.

```
Data-Science_projects/
|-- Healthcare-Infusion-Analytics/
|   |-- README.md
|   |-- docs/
|   |   `-- Healthcare_Infusion_Analytics_Documentation.pdf
|   |-- src/
|   |   |-- inspect_xlsb.py
|   |   |-- load_to_duckdb.py
|   |   |-- inspect_db.py
|   |   |-- check_joins.py
|   |   |-- explore_drugs_dates.py
|   |   |-- create_fact_infusions.py
|   |   |-- view_fact_infusions.py
|   |   |-- view_data_quality.py
|   |   `-- run_assessment_queries.py
|   |-- sql/
|   |   `-- assessment_queries.sql
|   |-- output/
|   |   |-- 01_unique_patients_by_month.csv
|   |   |-- 02_completed_treatments_q1_2026.csv
|   |   |-- 03_highest_drug_location_overall.csv
|   |   |-- 04_highest_drug_location_by_month.csv
|   |   |-- 05_weekly_operations_report.csv
|   |   `-- 06_data_quality_summary.csv
|   |-- requirements.txt
|   `-- .gitignore
```

9.1 What Not to Publish by Default

- The original assessment/source workbook unless redistribution is explicitly permitted.
- A database file that may reproduce source-level records unless the content is approved for public release.
- Any credentials, local paths, API keys, or environment-specific secrets.

PUBLIC PORTFOLIO PRACTICE

A strong public portfolio version can include the code, SQL, documentation, README, and approved/sanitized outputs. If the source workbook has redistribution restrictions, include a data/README.md explaining that the original source is intentionally excluded.

10. Assumptions, Limitations, and Analytical Cautions

Assumption / Limitation	Implication
appointments is treated as the infusion event table	All event-level measures inherit the appointment grain and source-system semantics.
meds_administered = 'Yes' defines successful treatment	Completion metrics are based on medication administration rather than appointment status alone.
No administered-volume field exists	Drug utilization uses count of administered infusion events as a proxy, not dose or milliliters.
Latest order series per patient used for drug mapping	Prevents appointment duplication but may not perfectly reconstruct the historically correct drug for every older event.
PIV attempts contains substantial missingness	Average PIV metrics should be interpreted only over non-null observations and may not represent every infusion.
Status and medication fields can disagree	The data-quality layer surfaces inconsistencies rather than overriding source values.

10.1 Important Drug-Mapping Caveat

Selecting the latest order series per patient is a pragmatic deduplication strategy supported by the available case structure, but it is not the same as a temporally exact appointment-to-order-series relationship. If a reliable event-level order-series key becomes available, the model should join at that grain instead.

11. Production and Portfolio Extensions

- Create a dimensional model with dim_patient, dim_drug, dim_location, and fact_infusions.
- Add dbt or SQL-based automated tests for uniqueness, null thresholds, accepted values, and relationship integrity.
- Build a Sigma, Power BI, or Tableau dashboard over the curated fact model.
- Add scheduling/orchestration so ingestion and reporting execute automatically when new source files arrive.
- Introduce event-time drug mapping when a trustworthy order-series linkage is available.
- Build treatment-volume forecasting and site-capacity monitoring.
- Add authorization lead-time and denial/approval workflow metrics if source fields support them.
- Create a data dictionary and metric-contract layer for business users.

12. Script Reference

File	Role in Workflow
inspect_xlsb.py	Profiles workbook sheets, columns, shape, and sample records.
load_to_duckdb.py	Loads every sheet into DuckDB after standardizing identifiers.
inspect_db.py	Lists DuckDB tables, schemas, and samples.
check_joins.py	Validates major joins, source distributions, and row multiplication risk.
explore_drugs_dates.py	Profiles drugs, date ranges, status/administration combinations, and authorization linkage.
create_fact_infusions.py	Builds the central fact_infusions analytical table.
view_fact_infusions.py	Displays fact rows for manual validation.
view_data_quality.py	Produces a focused data-quality inspection.
assessment_queries.sql	Contains the reusable analytical SQL definitions.
run_assessment_queries.py	Runs business queries and exports result CSVs.

13. Conclusion

This project demonstrates a complete analytical workflow rather than a collection of isolated queries: source profiling, ingestion, relationship testing, grain control, fact-table construction, reusable metric definitions, operational reporting, data-quality monitoring, and transparent documentation of assumptions. The result is a portable local analytics pipeline that can be reproduced, reviewed, and extended into a production-grade BI or data-engineering solution.